

部署與維運手冊

AI SPI Inference API

虛擬環境準備、離線 AIPC 搬運、設定、部署與疑難排解

涵蓋 Conda、uv、Python venv 與 Windows 服務化部署

文件屬性	內容
專案	ai-spi-inference-api 0.1.0
主要目標	無網際網路的 NVIDIA AIPC
Python	$\geq 3.12, < 3.13$
文件日期	2026-08-31
建議路徑	uv 鎖定檔 + 可攜 uv 快取；備援為 Conda Pack

本手冊以專案目前的 README、pyproject.toml、uv.lock、Windows 啟動器、設定驗證程式與環境檢查腳本為準。

如何使用本手冊

先看這裡 如果只需要一次成功部署，採用第 4 章的「建議方案 A：uv 離線快取」。若 AIPC 不允許安裝 Python 或 uv，改用第 7 章的 Conda Pack。不要直接複製現有 .venv。

閱讀導覽

- 第 1-3 章：確認專案架構、版本與部署前提。
- 第 4-7 章：分別準備 uv、venv、Conda 與離線移轉包。
- 第 8-11 章：在 AIPC 設定路徑、模型、TensorRT、啟動器與 Windows 服務。
- 第 12-13 章：驗收、上線、備份與回復。
- 第 14 章：依症狀查找 troubleshooting。

範圍與假設

- 遠端 AIPC 沒有網際網路，但可透過核准的 USB、內網檔案站或其他離線媒體取得部署包。
- AIPC 為 Windows 10/11 或 Windows Server x86-64，配備 NVIDIA GPU；Linux 可沿用依賴原則，但批次檔與 NSSM 章節不適用。
- SPI 機台共享資料夾可能仍透過廠內 LAN/SMB 提供；「離線」在此指無外網。若連 LAN 也沒有，watch_root 與輸出路徑需改成本機交換目錄。
- 模型權重不在目前 Git 檔案清單中，部署包必須另帶 .pt、.onnx 或 .engine 檔與其校驗碼。

重要限制 Python venv 官方定義為不可搬移、應在目標位置重建；TensorRT engine 也通常綁定平台、TensorRT 版本與 GPU。這兩類資產都不能假設「複製過去就一定能跑」。

1. 專案部署輪廓

正式部署建議使用三階段 durable pipeline。Ingest、Inference、Publisher 是獨立常駐程序，以 AIPC 本機 SQLite WAL 協調；模型服務則在 8000 與 8002 提供推論。相容 API 5050 是選配，不是三階段流程的必要元件。

元件	連接埠	責任	生產必要性
Publisher	-	到截止點接管、先發布 Primary、必要時至 23	必要，且先啟動
PatchCore anomaly	8000	輸出 anomaly_score	預設 required
Distance	8002	輸出 min_pad_distance	預設 required

元件	連接埠	責任	生產必要性
detection			
Inference	-	以最早截止期限執行模型並寫 manifest	必要
Ingest	-	監看、驗證、備份原始資料並排入 READY	必要，最後啟動
Paste detection	8001	paste_pixels；較重	預設停用
Compatibility API	5050	/process、/health、/ready	選配

啟動順序：Publisher -> 模型 8000/8002 且 health=healthy -> Inference -> Ingest。這不是批次檔編號順序；Publisher 必須先在線，因為它負責 deadline fallback。

2. 版本基準與不可省略的系統元件

層級	專案基準	離線包必備
Python	>=3.12,<3.13；啟動器只接受 3.12	Python 3.12 x64 安裝程式或已封裝 Conda 環境
PyTorch	2.7.1 + CUDA 12.8 index	torch 2.7.1+cu128 與 torchvision 0.22.1+cu128
ONNX Runtime	onnxruntime-gpu 1.19.0	相符 wheel 與 CUDA/cuDNN 執行庫
PyCUDA	2024.1	預建 wheel，或離線編譯工具鏈
TensorRT	10.x，但專案拒絕 10.1.x	與 driver/CUDA 相符的 NVIDIA 安裝包及 Python wheel
uv	pyproject.toml + uv.lock	相同版本 uv.exe、uv.lock、已暖機快取
Windows runtime	本機服務與原生 wheel	NVIDIA driver、VC++ Redistributable、必要時 VS Build Tools

目前工作區觀察 專案根目錄存在 uv.lock，且 requires-python=3.12；但 uv.lock 目前被 .gitignore 忽略，現有 .venv 則是 Python 3.10。部署包請明確包含 uv.lock，並在 AIPC 重新建立 Python 3.12 的 .venv。

2.1 AIPC 上線前資料表

- 作業系統版本與更新層級、CPU 架構（x86-64）、GPU 型號與 compute capability。
- NVIDIA driver 版本、CUDA runtime/toolkit、cuDNN、TensorRT 完整版本與 Python binding 版本。
- AIPC 專案固定路徑，例如 D:\Dre\JQ_SPI_02_AI_API；服務帳號與其檔案權限。
- SPI watch_root、machine return 目錄、備份目錄、本機 state/result/log 目錄。

- 模型檔名、格式、SHA256、input shape、batch profile、前處理契約與來源。
- 允許的輸入輸出媒體、惡意程式掃描流程、離線包簽核人與回復版本。

3. 三種虛擬環境方案如何選

方案	優點	限制	本專案建議
uv	快、使用 uv.lock、與 pyproject.toml 一致	離線端必須有完整快取與同版 uv	首選；在線同平台暖機後搬移 cache
venv + wheelhouse	只依賴標準 Python/pip、可稽核 wheel	需自行解決 PyCUDA/TensorRT 等原生套件	適合嚴格軟體白名單
Conda + conda-pack	可把 Python 與多數套件一起封裝	包大、同 OS/架構、首次解包後不可再搬	AIPC 無 Python/uv 時的可靠備援

建議決策 正式包以 uv 為主、Conda Pack 為備援；wheelhouse 同時保留做套件級修復。三者都要在與 AIPC 相同 OS、架構、Python minor 與 GPU 軟體基準的「在線孿生機」產生。

4. 建議方案 A：用 uv 準備可離線部署包

4.1 在線 Windows 孿生機準備

1. 安裝與 AIPC 相同的 Python 3.12 x64、NVIDIA driver/CUDA/TensorRT 基準；使用與目標端完全相同的專案路徑更容易重現問題。

2. 取得固定版本 uv.exe，建立獨立部署快取目錄，避免混入開發者其他專案的套件。

```
cd /d D:\build\ai_spi_inference_api
set UV_CACHE_DIR=D:\offline_bundle\uv-cache
uv --version
uv lock --check
uv venv --python 3.12 --clear
uv sync --locked --extra cuda --no-group dev
```

3. 如需在 AIPC 轉模型，額外同步 tensorrt-export；如需測試工具，再加入 dev。預設停用 paste，不要為生產核心包引入 Git 依賴。

```
uv sync --locked --extra cuda --extra tensorrt-export --no-group dev
rem 僅驗證包才加：uv sync --locked --extra cuda --group dev
```

4. 執行環境、GPU、provider 與測試驗證；成功後才封裝。

```
uv run python scripts\check_env.py
uv run python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available())"
uv run python -c "import onnxruntime as ort; print(ort.get_available_providers())"
uv run pytest tests\unit tests\integration
```

快取完整性 `uv --offline` 只會使用本機檔案與快取。要確認真的完整，請在學生機先中斷外網或封鎖網路，另建空白 `.venv`，使用相同 `UV_CACHE_DIR` 執行第 4.3 節命令。只做一次在線 `uv sync` 不等於已驗證可離線重建。

4.2 部署包目錄建議

```
AI_SPI_OFFLINE_2026-08-31\
project\           專案程式、uv.lock、設定範本、批次檔
uv\uv.exe          與暖機時完全相同版本
uv-cache\         專用且經離線重建驗證的 uv cache
installers\        Python 3.12、driver、VC++、CUDA、TensorRT、NSSM
wheelhouse\        修復用 wheels；含 CUDA torch 與原生套件
models\           模型、engine.json、SHA256SUMS.txt
config\ai_server.aipc.json
MANIFEST.txt
SHA256SUMS.txt
```

- 不要打包 `.venv`、`log`、`SQLite state`、`pipeline_results`、既有輸出或其他機台的 `config/ai_server.json`。
- 一定要明確帶入 `uv.lock`；本工作區的 `.gitignore` 會忽略它，單純 `git clone/zip tracked files` 可能漏掉。
- 所有檔案計算 SHA256；搬到 AIPC 後先驗證，再安裝或解壓。

4.3 AIPC 離線重建 uv 環境

5. 將部署包複製到 AIPC 本機磁碟，驗證 SHA256；先安裝 `driver`、`VC++`、`Python 3.12`、`CUDA/TensorRT`。安裝後重開機。
6. 把 `project` 放到固定路徑，`uv.exe` 與 `uv-cache` 放本機磁碟；不要從 USB 或 SMB 直接執行。

```
cd /d D:\Dre\JQ_SPI_02_AI_API
set UV_CACHE_DIR=D:\Dre\AI_SPI_OFFLINE\uv-cache
set UV_OFFLINE=1
D:\Dre\AI_SPI_OFFLINE\uv\uv.exe venv --python C:\Python312\python.exe --clear
D:\Dre\AI_SPI_OFFLINE\uv\uv.exe sync --frozen --offline --extra cuda --no-group dev --no-python-downloads
```

若 AIPC 也負責 `TensorRT` 轉換，將上一行加入 `--extra tensorrt-export`。若服務只跑既有 `.engine`，仍需 `TensorRT runtime/Python binding`，但不需要 `onnx/onnxslim`。

7. 固定啟動器使用的 Python，避免 PATH 檢到錯誤版本。

```
set PYTHON_EXE=D:\Dre\JQ_SPI_02_AI_API\.venv\Scripts\python.exe
%PYTHON_EXE% --version
%PYTHON_EXE% scripts\check_env.py
```

5. 方案 B：標準 venv + wheelhouse

這是最透明的離線安裝方式：AIPC 先離線安裝官方 Python 3.12，再就地建立 .venv，所有套件只從經核准的 wheelhouse 安裝。官方 Python 文件明確指出 venv 不應被視為可搬移資產，因此不要從學生機複製 .venv。

5.1 在線學生機建立 wheelhouse

8. 在完全相同的 Windows/Python/GPU 基準完成一次在線安裝；輸出實際解析版本。

```
py -3.12 -m venv D:\build\spi-wheel-build
D:\build\spi-wheel-build\Scripts\activate.bat
python -m pip install --upgrade pip wheel
python -m pip install torch==2.7.1 torchvision==0.22.1 --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements-cuda.txt
python -m pip freeze > offline_bundle\requirements-resolved-win-py312.txt
```

9. 把解析後的所有套件下載/建成 wheel。PyCUDA 若沒有相容 wheel，必須在具備 CUDA headers 與 MSVC Build Tools 的學生機先建好，不要把 sdist 留到離線 AIPC 才編譯。

```
python -m pip download -d offline_bundle\wheelhouse ^
-r offline_bundle\requirements-resolved-win-py312.txt ^
--extra-index-url https://download.pytorch.org/whl/cu128
python -m pip wheel -w offline_bundle\wheelhouse pycuda==2024.1
```

10. 另外放入 TensorRT 安裝包/whl；它未固定在 requirements-cuda.txt，需依 AIPC 的 NVIDIA 軟體矩陣選版。

供應鏈注意 混用 PyPI 與額外 index 時，應在受控學生機完成解析並保留檔案雜湊。離線 AIPC 一律使用 --no-index，避免任何未核准來源。

5.2 AIPC 離線安裝

```
cd /d D:\Dre\JQ_SPI_02_AI_API
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install --no-index ^
--find-links D:\Dre\AI_SPI_OFFLINE\wheelhouse ^
-r D:\Dre\AI_SPI_OFFLINE\requirements-resolved-win-py312.txt
```

```
.venv\Scripts\python.exe -m pip check
.venv\Scripts\python.exe scripts\check_env.py
```

若 PowerShell 不允許 Activate.ps1，不必調降全機安全政策；直接使用 .venv\Scripts\python.exe 呼叫即可。所有專案批次檔也支援用 PYTHON_EXE 指定完整路徑。

6. 方案 C：Conda 在線建立環境

專案附有 setup.bat，會建立 py312_cu128_j15 並在線下載 PyTorch CUDA 與 requirements.txt。它適合有網路的建置機，不適合直接在離線 AIPC 執行。

```
cd /d D:\build\ai_spi_inference_api
setup.bat
conda run -n py312_cu128_j15 python scripts\check_env.py
conda run -n py312_cu128_j15 python -m pip check
```

版本一致性 requirements.txt 是保留給 Conda/plain-pip 的 legacy mirror；依賴主來源仍是 pyproject.toml 與 uv.lock。建置前先比較差異，避免 Conda 包與 uv 生產包產生不同版本。

7. Conda Pack：把環境帶到無 Python 的 AIPC

7.1 在線學生機封裝

```
conda install -n base -c conda-forge conda-pack -y
conda pack -n py312_cu128_j15 ^
-o D:\offline_bundle\py312_cu128_j15.zip ^
--format zip
```

- 建置與目標必須是相同 OS 與架構；Windows 包不能搬到 Linux。
- 封裝前先在該環境驗證 torch/CUDA、ONNX provider、TensorRT import 與專案測試。
- Conda Pack 能處理環境前綴搬移，但 NVIDIA driver 仍屬系統層，不能只靠環境包。

7.2 AIPC 解包與固定路徑

```
powershell -NoProfile -Command ^
"Expand-Archive -Path D:\AI_SPI_OFFLINE\py312_cu128_j15.zip -DestinationPath D:
\Re\envs\py312_cu128_j15"
D:\Re\envs\py312_cu128_j15\Scripts\activate.bat
D:\Re\envs\py312_cu128_j15\Scripts\conda-unpack.exe
set PYTHON_EXE=D:\Re\envs\py312_cu128_j15\python.exe
%PYTHON_EXE% scripts\check_env.py
```

解包後不要移動 conda-unpack 完成後，該環境視為固定在目前路徑。若位置錯誤，刪除這個解包副本並從原始 zip 重新解到正確位置，不要再搬資料夾。

8. 專案、模型與設定的離線移轉

8.1 專案包應包含與排除

類別	包含	排除/另行處理
程式	app、scripts、tests、*.py、*.bat、README、pyproject.toml、uv.lock	.git、cache、開發機 .venv
設定	ai_server.example.json、AIPC 專用 config	其他機台的實際路徑與密碼
模型	PatchCore、center、pad、可選 paste；engine.json	來源不明或未驗證 artifact
運行資料	空白 state/log/result 目錄或由程式建立	舊 SQLite、舊 WAL、舊 job/result
系統安裝包	driver、VC++、Python/Conda、CUDA、TensorRT、uv、NSSM	與 AIPC 硬體不相符版本

8.2 SHA256 與部署清單

```
rem 在建置機產生 (PowerShell)
Get-Childitem D:\offline_bundle -Recurse -File |
  Get-FileHash -Algorithm SHA256 |
  ForEach-Object { "$($_.Hash) *$($_.Path)" } |
  Set-Content D:\offline_bundle\SHA256SUMS.txt
```

在 AIPC 逐項重新計算並比對。MANIFEST 至少記錄：包版號、Git commit、uv 版本、Python、torch/CUDA、onnxruntime providers、TensorRT、driver、GPU、模型 SHA256、設定檔 SHA256、建置與驗收人員。

信任邊界 TensorRT engine 與 Python pickle 型 .pt/.pth/.ckpt 都可能執行或載入可執行內容。只使用自建或經核准來源；PatchCore .pt 只有在可信時才加 --trust-pickle。

9. AIPC 專案設定

9.1 建立 AIPC 專用設定檔

11. 以 config\ai_server.example.json 複製出機台專用檔，例如 D:\Dre\config\ai_server.aipc01.json；不要直接覆蓋共享範本。
12. 以 AI_CONFIG_PATH 指向它，並讓所有模型/worker/相容 API 使用同一份經簽核設定。修改後必須重啟各程序，因為設定在每個 process 內快取。

```
set AI_CONFIG_PATH=D:\Dre\config\ai_server.aipc01.json
set PYTHON_EXE=D:\Dre\JQ_SPI_02_AI_API\.venv\Scripts\python.exe
```

設定欄位	部署原則	常見錯誤
paths.external_output_root	機台可見的 Primary 輸出；確認服務帳號可寫	NSSM 看不到使用者映射磁碟
paths.backup_output_root	AIPC 本機持久備份	放在 SMB 導致 SLA/可靠性降低
pipeline.watch_root	SPI timestamp 來源；durable Ingest 只看這個欄位	誤改頂層 legacy watch_root
pipeline.database_path	AIPC 本機 NTFS；所有 worker 共用同一檔	放 UNC/SMB 造成 WAL 鎖定
pipeline.staging_root	建議 null，直接用不可變 raw backup	多複製一次吃掉 30 秒預算
pipeline.result_root	AIPC 本機暫存 manifest/processed	目錄無寫入權限
model_clients	8000/8002 enabled+required；URL/欄位唯一	required service 未健康導致全 23
output	durable 必須 is_pass_only + machine_return	其他模式導致啟動驗證失敗
reliability	deadline 30、reserve 5；lease 必須符合驗證關係	reserve/lease 關係不合法
logging.log_dir	本機可寫；依 stage 產生 system.*	用相對路徑卻誤判位置

9.2 設定驗證命令

```
%PYTHON_EXE% -c "from pathlib import Path; from app.core.config import load_config;
c=load_config(Path(r'%AI_CONFIG_PATH%')); print(c.model_dump_json(indent=2))"
%PYTHON_EXE% -m app.pipeline publisher --once --config "%AI_CONFIG_PATH%"
```

第二行同時做 Publisher 所需路徑的 preflight；請先使用測試設定與空白測試目錄。若正式資料夾已有待處理工作，不要用 --once 做隨意測試。

9.3 建議的路徑與權限

```
D:\Dre\JQ_SPI_02_AI_API\    程式與 .venv
D:\Dre\config\              機台設定
D:\Dre\JQ_SPI_02_AI_API\state\ SQLite + WAL
D:\Dre\JQ_SPI_02_AI_API\pipeline_results\
D:\Dre\JQ_SPI_02_AI_API\log\
D:\Dre\JQ_SPI_02_AI_API\backup\
```

- 服務帳號對專案程式至少需讀取；對 state/result/log/backup 與 machine return 需修改/建立/重新命名權限。
- 若 external_output_root/watch_root 是 SMB，服務優先使用 UNC 並指定具權限的服務帳號；Windows service 通常看不到互動登入者的映射磁碟機。
- 不要把 SQLite database_path 放在 UNC/SMB；它必須在 AIPC 本機。

10. 模型與 TensorRT

10.1 選擇推論格式

格式	後端	AIPC 注意事項
.pt	PyTorch / Ultralytics	需 CUDA torch；PatchCore 可信 pickle 才可 --trust-pickle
.onnx	ONNX Runtime GPU	available providers 必須包含 CUDAAExecutionProvider；YOLO batch 動態或固定 8
.engine	TensorRT/PyCUDA	最快但最綁定；建議在目標 AIPC 建立並驗證

10.2 模型服務環境變數

```
set SPI_MODEL_DEVICE=cuda:0
set PATCHCORE_MODEL_PATH=models\patchcore\model_fp16.engine
set DISTANCE_CENTER_MODEL_PATH=models\distance\center.engine
set DISTANCE_PAD_MODEL_PATH=models\distance\pad.engine
set PATCHCORE_MODEL_ARGS=
start_model_services.bat
```

相對模型路徑以專案根目錄為基準；NSSM 的 AppDirectory 必須設為專案根目錄。若用可信 PatchCore .pt，再設定 PATCHCORE_MODEL_ARGS=--trust-pickle ... 並完整對齊訓練前處理。

10.3 在目標 AIPC 轉 TensorRT

TensorRT plan 預設只相容建立它的 TensorRT 版本、平台與 GPU 類型；本專案也明確要求在目標 NVIDIA AIPC 建 engine，且拒絕 TensorRT 10.1.x。輸入 artifact、profile、前處理與 batch 必須保留在 engine.json。

```
%PYTHON_EXE% -m pip install --no-index --find-links D:\AI_SPI_OFFLINE\wheelhouse ^
-r requirements-tensorrt-export.txt

convert_to_tensorrt.bat yolo ^
--input models\distance\pad.pt ^
--output models\distance\pad.engine ^
--task detect --precision fp16 --batch 8 --imgsz 640 640 ^
--workspace-gib 4 --save-onnx models\distance\pad.onnx
```

- center 與 pad 的 YOLO service 會以 batch 8 執行；動態 ONNX 必須提供涵蓋 1/最佳/8 的 profile，靜態圖則固定 8。
- PatchCore 要對齊 NCHW RGB float32 [0,1]、input size、center crop、ImageNet/custom normalization 與 score output。
- 轉換後用代表性 SPI 影像比較 PyTorch/ONNX/TensorRT 分數與最終 22/23 決策，不只看 engine 能否載入。

11. 部署與服務化

11.1 第一次手動啟動

13. 設定 PYTHON_EXE、AI_CONFIG_PATH 與模型路徑。先啟動 03 Publisher。
14. 執行 start_model_services.bat，輪詢 8000 與 8002；HTTP 200 不夠，JSON 內 status 必須是 healthy，且 backend/model/device 正確。
15. 模型健康後啟動 02 Inference；最後啟動 01 Ingest。

```
start "SPI Publisher" 03_pipeline_publisher.bat
start_model_services.bat
powershell -NoProfile -Command "Invoke-RestMethod http://127.0.0.1:8000/health | ConvertTo-Json -Depth 6"
powershell -NoProfile -Command "Invoke-RestMethod http://127.0.0.1:8002/health | ConvertTo-Json -Depth 6"
start "SPI Inference" 02_pipeline_inference.bat
start "SPI Ingest" 01_pipeline_ingest.bat
```

不要混跑 Durable mode 使用三個 app.pipeline worker；Compatibility mode 才用 scan_jobs.py + app.main。不要讓 legacy scanner 與 Stage 01 同時監看同一 share，否則可能重複發布。

11.2 NSSM 服務化

每個 worker 與模型各自建立服務，才能獨立重啟。以下示例省略模型參數；實際請把 model path/device 放入 AppParameters 或 AppEnvironmentExtra。

```
set REPO=D:\Dre\JQ_SPI_02_AI_API
set PYTHON_EXE=%REPO%\venv\Scripts\python.exe
set CONFIG=D:\Dre\config\ai_server.aipc01.json

nssm install SPI_03_Publisher "%PYTHON_EXE%" -m app.pipeline.publisher
nssm set SPI_03_Publisher AppDirectory "%REPO%"
nssm set SPI_03_Publisher AppEnvironmentExtra "AI_CONFIG_PATH=%CONFIG%"
nssm set SPI_03_Publisher Start SERVICE_AUTO_START

nssm install SPI_02_Inference "%PYTHON_EXE%" -m app.pipeline.inference
nssm set SPI_02_Inference AppDirectory "%REPO%"
nssm set SPI_02_Inference AppEnvironmentExtra "AI_CONFIG_PATH=%CONFIG%"

nssm install SPI_01_Ingest "%PYTHON_EXE%" -m app.pipeline.ingest
nssm set SPI_01_Ingest AppDirectory "%REPO%"
nssm set SPI_01_Ingest AppEnvironmentExtra "AI_CONFIG_PATH=%CONFIG%"
```

- 服務帳號需能讀寫 watch/output/backup/state/result/log；若使用 UNC，請用明確帳號，不要依賴登入者映射磁碟。
- 設定 stdout/stderr 旋轉與故障自動重啟；應用程式本身另寫 system.ingest、system.inference、system.publisher。
- 開機自動啟動不代表依賴已就緒。先啟動 Publisher，再讓模型健康後啟動 Inference/Ingest；可用 delayed start 或外部健康檢查腳本。

12. 上線前驗收

12.1 環境與 GPU

- [] %PYTHON_EXE% --version 為 3.12.x，sys.executable 指向預期環境。
- [] scripts/check_env.py 所有 required base package 為 OK。
- [] torch.__version__ 為 2.7.1 CUDA build；torch.cuda.is_available() 為 True。
- [] onnxruntime providers 含 CudaExecutionProvider（若使用 ONNX GPU）。
- [] tensorrt 與 pycuda 可 import（若使用 engine），driver/CUDA/TensorRT 版本已記錄。
- [] pip check 或 uv sync --locked --offline 可成功重建。

12.2 設定、目錄與服務

- [] AI_CONFIG_PATH 指向 AIPC 專用檔；Pydantic load_config 驗證成功。
- [] SQLite、backup、result、log 在 AIPC 本機；watch/output 權限以服務帳號驗證。
- [] 8000/8002 health 回報 status=healthy、模型路徑與 device 正確。
- [] Stage 03、模型、Stage 02、Stage 01 依正確順序啟動，沒有 legacy scanner 競爭。
- [] 所有模型與部署檔 SHA256 與 MANIFEST 一致。

12.3 端到端測試

16. 先以隔離的測試 config、watch_root 與 output_root 執行，不要直接對正式 SPI share。
17. 放入一個符合今日 14 位 timestamp 的小型 job；確認 CSV 欄位與所有期待影像存在、非空、可由 OpenCV 解碼。
18. 確認 SQLite 狀態依序到 READY / INFERENCING / RESULT_READY / PRIMARY_RETURNED / DONE。
19. 確認 Primary 先出現在 external_output_root，只有 is_pass 改變；原始編碼、delimiter、欄位順序、quoting、line ending 保留。
20. 確認 backup 下有 immutable raw、ai_result/returned、processed 與 manifest.json；log 記錄 latency 與 deadline_met。
21. 故意停止一個 required model，重跑隔離測試，確認 reason 為 required_model_failure/timeout 或 publish_reserve_reached，所有列安全回傳 23。

驗收門檻 正常路徑與 fail-safe 路徑都必須通過。這個系統的安全承諾不只是「AI 有結果」，也包含 required model 失敗或超時時仍能以全 23 發布。

13. 上線、備份與回復

13.1 正式切換

22. 凍結程式、uv.lock、模型與 config；記錄 SHA256 與版本。
23. 停止舊 scanner/舊 API 對相同 share 的自動處理；確認沒有殘留程序。
24. 備份舊版專案、環境、config、模型與必要 SQLite；不要覆寫唯一可回復版本。
25. 按 Publisher -> models -> Inference -> Ingest 啟動；監控首批 job 與 deadline。

13.2 回復策略

- 程式回復：切回上一個只讀版本目錄與其配套 config/模型/環境，不在原目錄混裝套件。
- 模型回復：.engine 必須與上一版 TensorRT/GPU 基準成套；不可只換單一 engine。
- 狀態回復：先停所有三個 worker，再處理 SQLite/WAL；不要在程序執行時手工複製或刪改資料庫。
- Primary 已發布的 job 具 at-least-once 行為；重啟可能以相同決策覆寫已回傳 CSV，這是設計內行為。
- 保留 manifest 與 stage logs，因 SQLite 是 job state，不是完整 audit event ledger。

14. Troubleshooting

先依序取得四類證據：目前使用的 Python、環境/GPU/provider、設定驗證、stage/model logs。不要先重裝全部套件，否則會抹去版本與錯誤線索。

```
where python
echo %PYTHON_EXE%
%PYTHON_EXE% --version
%PYTHON_EXE% scripts\check_env.py
nvidia-smi
```

14.1 啟動器顯示 Python 3.12 is required

症狀：resolve_python.bat 找到 .venv，但立即退出；或 uv 說 requires-python 不相容。

原因：環境不是 3.12。此工作區目前的 .venv 實際為 Python 3.10，是可重現的典型情況。

處理：停止使用該環境，改名保留後以 Python 3.12 重建；或明確 set PYTHON_EXE 指向正確環境。不要把舊 .venv 複製到 AIPC。

14.2 uv --offline 找不到套件或仍嘗試連線

症狀：cache miss、No solution found、failed to fetch，或離線重建失敗。

原因：uv-cache 未含目標 platform marker 的 wheel、uv.lock 漏包、uv 版本不同、Python/OS/架構不同，或 paste 的 Git 依賴未預先快取。

處理：回到相同 Windows x86-64 變生機，指定乾淨 UV_CACHE_DIR，使用相同 uv.exe 與 uv.lock 完成一次真正斷網重建；生產包不要啟用 paste，除非已把 Git commit 固定並完整快取/建 wheel。

14.3 pip --no-index 顯示 No matching distribution

症狀：某套件只有 sdist，或 wheel tag 不符 cp312/win_amd64。

原因：wheelhouse 在 macOS/Linux 或不同 Python 建立；PyCUDA/TensorRT 等原生套件沒有適配 wheel。

處理：在 Windows Python 3.12 x64 學生機重新下載/建 wheel；用 pip debug --verbose 檢查 tag。PyCUDA 需 CUDA headers/MSVC；TensorRT wheel 從相符 NVIDIA 安裝包取得。

14.4 DLL load failed / 找不到指定模組

症狀：import torch、onnxruntime、tensorrt、pycuda 或 cv2 失敗。

原因：VC++ runtime、CUDA/cuDNN/TensorRT DLL 缺失，PATH 順序錯，或 32/64 位元不一致。

處理：檢查 nvidia-smi、where 對應 DLL、系統 PATH 與安裝版本；重開機後再測。不要靠把不明 DLL 複製進專案目錄解決。

14.5 torch.cuda.is_available() = False

症狀：torch 可 import，但模型使用 cuda:0 失敗。

原因：裝到 CPU wheel、driver 不支援、GPU 不可見，或服務帳號環境不同。

處理：確認 torch.__version__ 含 +cu128、torch.version.cuda、nvidia-smi；以實際服務帳號執行相同命令。必要時從核准 cu128 wheel 重建環境。

14.6 ONNX Runtime 沒有 CUDAExecutionProvider

症狀：available providers 只有 CPUExecutionProvider；專案在 cuda:N 啟動時拒絕默回 CPU。

原因：裝到 onnxruntime 而非 onnxruntime-gpu，或 CUDA/cuDNN DLL 不相容。

處理：移除 CPU/GPU 套件衝突，離線安裝 onnxruntime-gpu==1.19.0，重查 provider 與 native DLL。不要以 CPU fallback 當作正式修復。

14.7 TensorRT engine 無法反序列化

症狀：版本、magic tag、compute capability、binding/profile 或 engine validity 錯誤。

原因：engine 在其他 OS/GPU/TensorRT 建立、runtime 升級、檔案損壞，或用到 10.1.x。

處理：核對 engine.json 與 SHA256；在目標 AIPC 用核准的 TensorRT 10.x（非 10.1.x）重新轉換，確認 batch/profile 與 preprocessing。

14.8 模型 /health 回 200，但還不能處理

症狀：HTTP 成功，JSON status 卻是 initializing/unhealthy，或模型資訊不符。

原因：TensorRT engine 仍初始化、模型路徑錯、GPU OOM，或 backend 載入失敗。

處理：以 payload 的 status=healthy 為啟動門檻；查看模型視窗/stderr，確認 model path、device、free VRAM，再啟動 Stage 02/01。

14.9 Worker 一啟動就退出 / Config invalid

症狀：pipeline.enabled、路徑 preflight 或 Pydantic validation error。

原因：watch_root 空白、沒有 required model、duplicate name/target_column、deadline/reserve/lease 關係不合法，或 output 模式不符 durable contract。

處理：用第 9.2 節 load_config 命令讀完整錯誤；durable 必須 is_pass_only + machine_return，worker_lease > deadline，publisher heartbeat < lease < reserve。

14.10 Ingest 一直沒有 READY

症狀：timestamp folder 被反覆掃描但不進 READY。

原因：不是今日 14 位資料夾、settle 期間持續變動、CSV 無資料或缺欄、期待影像缺失/空檔/不可解碼、影像 key 重複。

處理：查看 system.ingest；核對 component_name、Array_id、Pad_id 與 image_name_template。歷史資料用 --once --date YYYY-MM-DD，並在測試設定執行。

14.11 結果全部是 23

症狀：API 仍可能回 200，但輸出全部 fail-safe 23。

原因：required model failure/timeout、回傳 key 不完整/非有限數值、超過影像門檻，或 publish reserve 到點。

處理：讀 backup/.../ai_result/manifest.json 的 reason/errors/timing，再查 system.inference 與模型 logs。這通常是安全策略正常啟動，不應先改成 optional。

14.12 SQLite locked / 狀態不可靠

症狀：鎖定錯誤、WAL 異常、worker 看見不同 queue。

原因：database_path 在 SMB/UNC、各 worker 指到不同 config/database，或防毒/備份程式干預。

處理：移到 AIPC 本機 NTFS，確認所有服務的 AI_CONFIG_PATH 一致；排除即時備份掃描。維修前先停止全部 worker。

14.13 Job 卡在 PUBLISHING 或 PRIMARY_RETURNED

症狀：長時間未 DONE。

原因：PUBLISHING 多為 machine return/SMB 寫入重試；PRIMARY_RETURNED 表示 Primary 已可見，只剩本機 ai_result finalize。

處理：查看 system.publisher、權限、磁碟空間與 share。修復後讓 Publisher 依 lease 自動回復；不要重跑 inference 或手改 Primary。

14.14 NSSM 服務找不到磁碟、模型或設定

症狀：手動可跑，服務模式卻 FileNotFound/Permission denied。

原因：AppDirectory 未設、相對路徑基準錯、服務帳號無權限、映射磁碟不存在、AppEnvironmentExtra 不完整。

處理：固定 AppDirectory、使用完整 PYTHON_EXE/AI_CONFIG_PATH/model path；SMB 改 UNC 並指定服務帳號。以該帳號做讀寫測試。

14.15 Primary 寫入失敗或超過 30 秒

症狀：deadline_met=false、share timeout、Publisher 重試。

原因：SMB 不可用、原始備份過慢、GPU/CPU 過載、staging_root 造成第二次複製，或模型超過 publish cutoff。

處理：先修 share 與本機 IO；staging_root 保持 null；檢查 ready_at 到各 stage latency。30 秒是 soft real-time，AIPC/SMB 中斷時無法保證。

14.16 high cover 規則永遠不觸發

症狀：其他缺陷正常，但沒有 high cover。

原因：paste 8001 預設停用；cover% 還需要 paste_pixels 與由 Width/Length 得到的 pad_area。

處理：只有在確定需求後才離線打包 MobileSAM/paste extra、啟動 8001、enabled=true 並重啟相關程序；驗證新增負載不破壞 30 秒預算。

15. 快速命令表

15.1 版本與環境

```
%PYTHON_EXE% --version
```

```
%PYTHON_EXE% -c "import sys; print(sys.executable); print(sys.version)"
```

```
%PYTHON_EXE% scripts\check_env.py
%PYTHON_EXE% -m pip check
%PYTHON_EXE% -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available())"
%PYTHON_EXE% -c "import onnxruntime as ort; print(ort.__version__, ort.get_available_providers())"
%PYTHON_EXE% -c "import tensorrt as trt; print(trt.__version__)"
```

15.2 Worker 診斷

```
01_pipeline_ingest.bat --once --date 2026-08-31
02_pipeline_inference.bat --once --worker-id manual-inference
03_pipeline_publisher.bat --once
```

使用條件 `--once` 仍會對設定中的資料庫與目錄採取實際動作。只在隔離測試 config 或確認沒有正式待處理 job 時使用。

15.3 健康檢查

```
Invoke-RestMethod http://127.0.0.1:8000/health | ConvertTo-Json -Depth 6
Invoke-RestMethod http://127.0.0.1:8002/health | ConvertTo-Json -Depth 6
Invoke-RestMethod http://127.0.0.1:5050/ready | ConvertTo-Json -Depth 6
```

5050 /ready 只屬相容 API，並不代表三個 durable worker 的 heartbeat/queue 都正常；stage 狀態仍需看 SQLite、system.* logs 與 manifest。

16. 官方與專案參考

- 專案 README.md：專案根目錄 README.md
- 專案 pyproject.toml / uv.lock：依賴、Python 範圍、平台 marker 與鎖定版本
- 專案 config/ai_server.example.json：設定範本
- Python 3.12 venv：<https://docs.python.org/3.12/library/venv.html>
- uv CLI：offline / frozen / no-python-downloads：<https://docs.astral.sh/uv/reference/cli/>
- uv cache：<https://docs.astral.sh/uv/concepts/cache/>
- pip download：https://pip.pypa.io/en/stable/cli/pip_download/
- Conda Pack：<https://conda.github.io/conda-pack/>
- NVIDIA TensorRT Engine
Compatibility：<https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/engine-compatibility.html>

- NVIDIA TensorRT Support

Matrix : <https://docs.nvidia.com/deeplearning/tensorrt/latest/getting-started/support-matrix.html>

文件維護建議

- 每次更新 Python、CUDA、PyTorch、ONNX Runtime、TensorRT、模型或設定 schema 時，重做離線重建與端到端驗收。
- 把 uv.lock 納入正式版本控制或部署產物清單；目前只存在工作區且被 .gitignore 忽略，容易在交付時遺漏。
- 將每台 AIPC 的版本矩陣、模型 SHA256、config SHA256、驗收結果與回復包位置納入變更單。